

Fast, without being wasteful

Resource efficiency audit of tiptotyp

The app has a credible performance foundation, but it is not yet deliberately memory-budgeted. It is better at **avoiding repeated computation** than at **limiting the amount of retained data**. The next step should be bounded working sets and explicit resource lifetimes, not removing useful features or rewriting the UI.

My assessment: good caching and repaint discipline; substantial opportunities in raster-preview memory, undo storage, per-window service ownership, and idle polling. There is not enough evidence to claim that the GPU itself is the bottleneck, or that every retained allocation is a leak.

What the current measurements say

In isolated, optimized, capture-based runs on an M2 Max with 32 GiB unified memory:

- One small document used **448 MiB in the app**, or **497 MiB including directly attributable child processes**.
- Four document windows used **723 MiB in the app / 925 MiB in that process family**.
- After closing the document windows, child processes and editor passes stopped, but the app still had a **409 MiB physical footprint**.
- A simple twenty-page preview used **666 MiB in the app**, versus **417 MiB for one page**. That is a 249 MiB increase, despite only the first pages being visible.

These are snapshots, not minimum system requirements. The capture workflow exercises raster preview, and some native WebKit helper processes cannot be attributed by parent PID. The reported family is therefore not an exhaustive account of the app's resource cost. Full methodology and limitations follow.

What I would do first

1. Account for decoded-image and texture bytes, then keep only visible preview pages plus a small nearby cache at useful resolution.
2. Audit cache cleanup on window close; distinguish the required native host from document, font-preview, and graphics data that can be released.
3. Stop reading whole files and rescanning workspaces on short fixed timers.
4. Give undo and background work byte/concurrency budgets, not just item counts.

Scope: source review plus seven completed runtime scenarios. This report changes no application behavior. Suggestions below are proposed work, not completed fixes. Source references identify the local checkout audited, not external documentation.

1. Where the resources go

The useful unit of accounting is the **whole application family**, not just the process named tiptotyp. A quiet UI can still have a busy compiler; a small Rust heap can coexist with large graphics surfaces or a language-server cache.

Owner	Resources and lifetime
Native app / egui	Windows, font atlases, editor buffers, parse/layout caches, raster RGBA, textures, worker threads and shared settings.
Each document window	Its active document and parked tabs, preview state, compiler/asset services, Tinymist session and workspace/index state.
Typst / Tinymist	Compilation and language intelligence in child processes. Their memory must be measured alongside the UI.
Native web preview	WebKit view and associated OS-managed helper processes, page/rendering caches and shared graphics surfaces.
Disk	Private compiler workspaces, PDF/PNG intermediates, installed toolchain, settings and profiling evidence. Build output is a separate developer cost.

Four distinctions that matter

Memory footprint is not virtual address space. A reserved thread stack or mapped file is not necessarily occupying physical RAM. This audit uses macOS physical footprint snapshots; it does not interpret the much larger virtual-memory total as RAM use. Footprint, resident memory, dirty memory and compressed memory are different measurements and should not be added together.

GPU memory is not a second independent pool on this machine. The M2 Max uses unified memory. A texture-byte estimate describes a graphics resource; it should not simply be added to the measured process footprint. Driver padding, backing stores, sharing and compression prevent an exact conversion from pixel counts.

Fast caches are useful; unlimited caches are not. Keeping recently viewed pages can make scrolling instant. Keeping every page at a fixed high resolution is a different policy. The goal is to retain the working set that buys responsiveness and release the rest under a predictable budget.

Low average CPU does not prove smooth interaction or low energy. Short pauses can matter despite a low average. Timed waits can wake frequently without using much CPU. This audit did not measure watts, GPU execution time, or scrolling latency percentiles, so it makes no battery-life or frame-rate claims.

A sound foundation to preserve

The code already uses deferred, independently repainted windows; revision-keyed derived data; shared syntax databases; and background compilation and asset decoding. Parked tabs store documents and editor state rather than entire app instances. These are the right directions. More threads everywhere would not automatically use fewer resources, and native UI ownership must remain safe.

Sources: `src/app/tabs.rs:6`; `src/windowing/document_host.rs`; `src/viewport_fonts.rs:18`; `src/editor_data.rs:79`; `src/generic_highlight.rs`.

2. Measured current state

Scenario	App MiB	Family MiB	App CPU	Editor passes / 10 s
Small document	447.7	496.8	1.08%	52
Settings open	441.9	466.2	1.62%	49
Four windows	722.9	924.9	4.22%	181
All windows closed	408.9	408.9	0.24%	0
5,000 comment lines	438.8	474.8	3.39%	101
One simple page	417.3	443.1	2.35%	195
Twenty simple pages	665.8	698.8	1.65%	175

CPU is an approximate app-process percentage of one logical core, derived from CPU-time deltas across 4.25-4.72 seconds, while native sampling was active. It is not GPU utilization or a percentage of all twelve cores. “Family” sums app and observed descendant footprints: respectively 3, 2, 9, 1, 3, 3 and 3 processes.

Interpretation, not a benchmark contest

Idle caching works. All seven completed runs recorded zero syntax-highlight rebuilds during their ten-second measurement periods. Small-document editor passes averaged 673 microseconds of inclusive wall time. Settings itself recorded ten `ui.settings` calls averaging 2.19 ms. Nested spans are not additive CPU time.

Idle is not completely event-driven. The small document still had 52 editor passes in ten seconds.

Recorded repaint requests point to external-file and Explorer polling, plus worker completion. The small synthetic page fixtures had more passes, so a universal “idle FPS” cannot be inferred from one fixture.

Window count matters. Four windows had a 925 MiB attributable family footprint, compared with 497 MiB for the small single-window case. This includes additional sidecar processes, not merely four copies of the visible UI. It is not a prediction that every new window adds an identical amount.

Closing windows saves activity, but not all memory. The dormant host had no editor passes or descendant processes in the sample. Its retained 409 MiB warrants allocation/lifetime profiling. A single post-close reading does not demonstrate an ever-growing leak or prove which subsystem owns that memory.

Do not overinterpret small differences. Settings and the large-comment fixture had slightly lower footprints than the small main fixture. Backend startup state, compression and run variation differ; this does not mean adding settings or text reduces memory. The one-to-twenty-page increase is supported by a concrete all-page retention path, but its exact size still needs repetitions.

Evidence: `resource-audit-evidence/results.json`; each successful run retains `metadata.json`, `footprint.json`, `vmmmap.json`, CPU snapshots, native stacks and `summary.json`. Numbers rounded to 0.1 MiB; 1 MiB = 1,048,576 bytes.

3. Preview: the largest clear lever

What is happening now

Established in source: the raster path asks Poppler to render the entire PDF at 144 DPI. It decodes every page into RGBA before returning the page vector. The UI then uploads every page as a texture and keeps the original RGBA bytes. Changing light/dark page appearance rebuilds those textures. The raster view iterates all pages and their link geometry even when most are outside the viewport.

Keeping an old successful preview while a new build runs is good UX. However, old pages, new decoded pages, temporary conversion buffers and texture uploads can overlap. A steady-state footprint therefore understates the possible peak. Directly opened PDF assets also use all-page rasterization.

Scale estimate, not measured VRAM: an A4 page at 144 DPI is approximately $1,191 \times 1,684$ pixels. At four bytes per pixel, that is about 7.65 MiB per representation. Twenty pages need about 153 MiB for raw RGBA alone; a nominal second texture representation brings this to about 306 MiB. One hundred pages approach 1.49 GiB for those two representations, before other costs.

This estimate explains why a visually simple document can be expensive: a nearly blank raster page has the same dimensions as a dense page. OS compression can change physical footprint, but it does not remove the app's ownership of buffers.

How to reduce it without making scrolling worse

1. Keep page dimensions and offsets for the whole document, but create widgets, decoded pixels and textures primarily for the visible range.
2. Prefetch a small number of neighboring pages in the scroll direction. Use a byte-bounded least-recently-used cache shared across windows, with active pages protected from eviction.
3. Render at a resolution appropriate to actual display size and scale. Retain a lower-resolution page while a zoomed replacement arrives. PDF export remains full quality and independent of this screen-preview budget.
4. Avoid full-document CPU color conversion on theme changes. First limit work to resident pages; then assess a shader or other draw-time transform if fidelity and profiling justify it.
5. Bound concurrent raster jobs and cancel obsolete work between pages. Admit work by estimated pixel bytes, not just number of requests.

Halving DPI quarters pixel count, but globally lowering quality is not the first recommendation. The better target is **not rendering invisible pixels**. A renderer rewrite should follow evidence, not precede this ownership change.

Graphics measurement still missing

The small-document vmmap snapshot reported 109.5 MiB resident in IOSurface regions and 26.7 MiB in IOAccelerator regions. These are useful graphics-backing clues, not exact dedicated GPU memory, and must not be added to footprint. GPU time, driver allocation lifetimes and attributed WebKit helpers remain unmeasured.

Sources: `src/compiler.rs:27,820,947`; `src/app.rs:2392,5979,8750`; `src/preview.rs:24`; `src/app/raster_view.rs:80`; `core/src/preview.rs`; `src/asset.rs:44`. Measurement: `main run vmmap.json`, page-count runs.

4. Text, undo and small caches

Undo is count-bounded, not byte-bounded

DocumentSession holds a mutable source string, an immutable snapshot and the saved source. Undo retains up to 100 full previous source snapshots. Sharing an Arc avoids copying the same revision between readers; it does not share unchanged substrings between different revisions.

Illustrative scaling: 100 distinct versions of a 5 MiB document can retain about 500 MiB of source history in one tab, before current text, saved text, parsing and layout. This was established from ownership, not measured with a 500 MiB heap test. Multiple parked tabs preserve their histories, so window-only accounting misses it.

Introduce byte accounting and an explicit history policy. A byte cap is relatively simple, but truncating history changes what the user can undo and must be deliberate. If large-document editing is common, investigate edit deltas or a persistent text structure. Never evict unsaved current content to satisfy a cache budget.

Cache hits still do some whole-document work

Derived editor data is revision-keyed, and Typst reparsing is incremental on a change. Both are strengths. However, the highlighter constructs a source string before its hit check and clones its cached layout job on a hit. A cached parse is therefore not equivalent to a zero-allocation frame. The character-offset index also stores a machine-sized offset per Unicode scalar, alongside source and parse representations.

Use revision/identity keys before constructing temporary strings where possible; share immutable layout data where the UI API permits it. Measure allocation bytes per unchanged frame before replacing text structures. Preserve the disabled miTeX path's no-projection-work behavior; optional modes should not add work for documents that do not use them.

Small caches: good local limits, incomplete lifetime policy

Cache	Good today	Remaining issue
Asset hover images	8 entries; longest edge capped at 720 pixels. PDF hover renders only page one.	Image input can be fully decoded before downsampling; final thumbnail size does not bound peak decode memory.
Tooltip code layout	32 cached jobs; avoids repeated highlighting.	Entry count does not bound source/layout bytes.
Font samples	4 samples per slot; private atlas capped at 1,024 pixels per side; font file size limited.	Global map is keyed by viewport and slot; no explicit closed-viewport removal was found.

The font-sample map is a concrete ownership risk: slots retain texture handles even though each slot is locally bounded. Add viewport-close cleanup and test repeated open/preview/close cycles. This is a source finding, not a demonstrated monotonic leak in a long-running measurement. Avoid “clear everything every frame”: it would replace retention with repeated expensive rasterization.

Sources: core/src/document.rs:112,360,380; src/editor_data.rs:181; src/highlight.rs:98; src/app/tabs.rs:6; src/font_preview.rs:49,170; src/app/tooltips.rs:1389; src/asset.rs:40,358; src/mitex_projection.rs.

5. Background activity and lifecycle

Replace routine polling with meaningful events

Every second, the active editable file is read in full and fingerprinted on the UI thread. Explorer refresh also walks the workspace recursively every two seconds while visible. Its exclusions are `.git`, `target` and `.tiptotyp`, not a general ignore policy. Large dependency trees can therefore make an otherwise idle project costly. Hiding Explorer suppresses its scan, but not the external-file check.

For four windows each editing a 10 MiB file, the file check alone can read about 40 MiB of logical data per second. That is an algorithmic estimate, not measured physical disk traffic: the OS may serve it from cache. Either way it performs avoidable work and can block the UI on slower storage.

Use filesystem events, metadata filtering and coalesced workers; retain a slower verification fallback for missed events. Share watches/index snapshots by canonical workspace where ownership permits. Support appropriate workspace exclusions. Preserve correctness for atomic saves, deletes and changes that metadata alone cannot distinguish. Git's already event-driven refresh is a good model.

Background does not mean free

Each window constructs compiler, asset-loader, thumbnail-loader and Tinymist services. Active compiler and Tinymist loops use 15 ms and 10 ms receive timeouts, respectively. Together that permits roughly 167 timeout opportunities per second per active pair; this is not a measured wakeup count or power estimate. Native samples show the timed waits. Asset queues block when idle, which is preferable.

Use event multiplexing or a shared wake mechanism where feasible, while keeping response latency and shutdown deadlines. Lazily create unused services. Consider sharing immutable project data before attempting to share a stateful language server across windows with potentially different unsaved buffers.

The latest-only mailbox bounds pending replaceable requests, and stale-result tokens prevent old work overwriting new state. But `LatestJob` starts a fresh thread and discards the old receiver; it does not interrupt the old computation. Rapid replacement can overlap obsolete jobs. A bounded executor plus cooperative cancellation would bound total CPU/memory demand. Never apply latest-only semantics to Git, save or other mutations that must complete.

Tinymist's individual messages have a 32 MiB limit, but its channels are unbounded in aggregate. Consider coalescing replaceable updates and limiting queued bytes. This is a burst-risk finding; no overflowing queue was observed in these idle runs.

Hidden, closed and gone are different states

Hiding a WebKit view retains it for quick return. Closing a document explicitly stops its sessions and drops its webview. macOS also retains a root native host to support reopening windows safely. Keep these distinctions: immediately destroying everything can regress resume latency and the previously fixed native-context crash.

Audit disposable cache owners in the dormant host, and add an idle/memory-pressure release policy with a warm-resume path. Repeated native bounds/background/visibility assignments are a smaller optimization opportunity: cache unchanged properties, but preserve DPI, focus and clipping correctness.

Sources: `src/app.rs:123,1366,1480,1710,4930`; `src/workspace.rs:240,302`; `src/compiler.rs:28,459`; `src/tinymist.rs:29,41,660,1581`; `src/worker.rs:69`; `src/worker/latest_queue.rs`; `src/app/native_views.rs:1349,1386`.

6. A resource-conscious work plan

The aim is a predictable working set at the same useful responsiveness. The order below balances confidence, expected impact and implementation risk. No percentage savings are promised without before/after measurements.

Order	Work	Evidence and acceptance test
1	Resource accounting + raster working set	Highest-confidence scaling issue. Track decoded/texture bytes; test 1/20/100 pages at equal viewport and zoom. Memory should follow resident pages, not total pages; scrolling and zoom must stay responsive.
2	Viewport/cache cleanup	Explicit font-slot lifetime gap; large dormant footprint observed. Repeat window/font-preview open-close cycles; live resource counts must return to a baseline or plateau. Attribute allocations before claiming a leak fix.
3	File/workspace event handling	Whole-file and whole-tree polling found in source. Test no periodic full reads for unchanged files; one coalesced scan per workspace; reliable external edits.
4	Undo/storage budgets	Full snapshots with a 100-entry cap. Test large edits across many tabs; enforce documented byte policy without losing current content.
5	Bound jobs and service activity	Per-window timers and non-cancelled superseded jobs. Test bounded concurrent reads/rasterizers, bounded queue bytes and no unnecessary idle wakeups.
6	Copy/native-call reductions	Cached-layout copying and repeated native property updates. Compare allocation counts and native calls under an identical interaction trace.

Put limits where the expensive objects are

Track current and peak bytes for decoded pages, nominal textures, thumbnail caches, font previews, retained document revisions and queued jobs. Count shared allocations once. Record active workers, child processes, raster jobs and cache hit/miss/eviction counts. Keep this opt-in, bounded and content-free; do not write logs every frame.

As **experimental starting points**, try visible pages plus two neighbors and a process-wide 128 MiB nominal texture / 256 MiB decoded-page budget. These are not validated defaults: high-DPI and multi-window workloads need testing and active-page pinning. A process-wide limit matters because a per-window cap multiplies with windows.

Use distinct policies for reusable caches and user history. A cache can be rebuilt; discarded undo cannot. Track undo bytes first, then select a user-visible policy or change its storage representation. Do not conceal history loss as an optimization.

Avoid attractive but premature changes

Do not disable the GPU, remove all animations, destroy every hidden preview, or move every window onto its own rendering thread without measurements. None has been established as the best resource tradeoff here. Likewise, do not start a renderer rewrite before fixing all-page retention, or sacrifice export quality to reduce an on-screen cache.

7. What to keep, and how to verify

Existing work that is paying off

- **Demand-driven repainting and window isolation:** zero editor passes in the no-window run; deferred child windows avoid redundant immediate atlas copying.
- **Revision-aware computation:** no highlight rebuilds in the measured idle periods; incremental Typst parsing, shared syntax/theme data and cached settings state.
- **Background slow operations:** compilation, image decoding and PDF rasterization are not performed wholesale inside the UI drawing callback.
- **Bounded replaceable requests:** latest-only mailboxes, cancellation tokens and stale-result rejection avoid useful classes of duplicated and outdated work.
- **Selective expensive paths:** interactive preview can avoid CLI raster work when it is not required; thumbnails use the first PDF page; optional miTeX projection is guarded. Preserve these paths rather than introducing unconditional work.
- **Testability and observability:** the profiling runner records fixtures, phases and bounded spans; geometry/state tests offer reliable regression checks.

Make resource efficiency an acceptance criterion

For each optimization, record the same optimized binary profile, fixture, viewport, theme, warmup and interaction sequence before and after. Repeat runs and report variation. Keep four workloads separate:

1. **Idle:** app plus sidecars, UI passes, wakeups and file operations over a longer interval. Include settings, hidden preview, minimized windows and all closed.
2. **Active:** typing, scrolling with hover cards, theme changes, tab switching and preview zoom. Record tail frame latency, CPU/GPU work and allocation rate.
3. **Cold/miss:** first opening, font discovery, large image decode, new page render and large workspace indexing. Record peak footprint and first-use latency.
4. **Warm/hit:** revisit the same pages/tabs and reopen a recently hidden preview. Verify that memory savings do not turn every interaction into a cold miss.

Add scaling cases for 1/4/8 windows, 1/20 tabs, 1/20/100 pages, larger sources, large images and large dependency trees. Include repeated open-close cycles and memory-pressure recovery. Use deterministic assertions for byte limits, cache reuse, queue bounds and cancellation; avoid flaky CI wall-time thresholds.

Disk and energy deserve separate ledgers

This checkout had about 50 GiB in target, 146 MiB in toolchain, 3.2 MiB in assets, and 216 MiB in .tiptotyp when inspected. These are local development directory sizes, not the installed application's RAM or distribution size. Debug/profiling artifacts and capture history should have an explicit developer retention policy; no files were deleted in this audit. Tempfile-managed raster intermediates already have automatic lifetime cleanup.

Energy requires a quiet, repeatable desktop workload and native energy/wakeup measurement. This audit cannot establish watts saved from CPU percentages alone. The safest first energy improvements are eliminating unnecessary work, reducing pixel traffic, and letting idle workers sleep until an actual event.

Further local context: docs/performance.md; docs/multi-window-audit.md; src/viewport_fonts.rs; src/worker/latest_queue.rs; src/app.rs:5825. The existing multi-window document contains historical measurements; they are not used as before/after numbers for this audit.

8. Method, evidence and limits

Environment. Apple M2 Max, 12 logical CPU cores, 30 GPU cores, 32 GiB unified memory; macOS 14.6.1 (23G93), arm64; built-in Retina display. Rust 1.96.0. Audit date: 17 September 2026. Source HEAD: 27f644cf8752fa75731e9003350f26f245016d6e. Each run records source-file hashes, Git status, binary and pinned toolchain hashes to identify the actual tested state.

Build. The app used an optimized profiling build with frame pointers:

```
cargo build --locked --profile profiling --features profiling \
  --bin tiptotyp \
  --config 'build.rustflags=["-C", "force-frame-pointers=yes"]'
```

Workload. Each successful scenario launched a fresh process in an isolated workspace, with non-persistent QA settings and Catppuccin Latte. The main window requested 1,400 × 900 logical points, constrained by the display; captured main framebuffers were approximately 2,800 × 1,770 pixels. The small fixture is docs/ui-snapshots/theme-fixture.typ. The larger source is 5,000 generated Unicode comment lines. The page-count fixtures contain one heading and sentence per A4 page. No typing or scrolling was injected in these idle measurements.

After the capture-ready marker, the app warmed for eight seconds and recorded ten seconds of spans. The collector took footprint and vmmap snapshots, then bracketed a four-second native sample with process CPU-time snapshots. Startup and peak compile memory were not measured. Sampling and memory inspection are intrusive, and other user applications remained running. There was one completed trial per scenario; no statistical confidence interval or before/after optimization claim is implied.

Important capture limitation. The deterministic scene exercises raster output even when production interactive preview can avoid it. Retained buffers, surfaces and a CLI watcher may reflect that warmup. These results describe the named capture-based workload, not the minimum possible production footprint. Native WebKit child views are not composited into the egui framebuffer; inspected PNGs confirm fixture content, not whole-window native composition.

Process attribution. Descendants were identified recursively by parent PID. OS-managed WebKit processes with another parent were retained as unattributed candidates, not added to totals. No exact GPU utilization, dedicated VRAM, energy, allocation-stack census or exhaustive WebKit process-family accounting was obtained. No cross-platform resource guarantee follows from this macOS run.

Failed attempts are retained. Initial direct-PDF launches did not reach the main scene's capture-ready condition within 120 seconds and were terminated by the collector. They are excluded from results. The successful pdf-1 and pdf-20 scenarios open their generated Typst sources; the names refer to rendered page counts, not successful direct-PDF-tab profiling. No failure cause in the product is asserted from the capture timeout.

Reproduction. Run `python3 output/pdf/resource-audit-evidence/collect.py main (or settings, multi-window, no-window, large, pdf-1, pdf-20)`, then `python3 output/pdf/resource-audit-evidence/summarize.py`. Each run creates a new timestamped evidence directory; raw observations and fixtures are retained. Compile this report with `typst compile output/pdf/resource-audit.typ`.

Bottom line: keep the existing responsiveness mechanisms, but put byte budgets and lifecycle rules around their retained state. The strongest first investment is a bounded preview working set, followed by idle polling and window/cache lifetime cleanup. Measure the whole family and preserve fast cache hits.